

RAG-based AI chatbot

Goal of the Prototype:

The goal of this prototype is to develop an **AI-powered mental health chatbot** integrated into the **mobile application** where patients can comfortably talk about their mental health concerns. The chatbot will be based on the **Retrieval-Augmented Generation (RAG)** approach, utilizing knowledge derived and paraphrased from the **DSM-5-TR** (used only as a reference source). It will provide supportive, informative, and empathetic responses through natural language interaction.

The **chat history** between the patient and the chatbot will be securely stored in the backend and later analyzed to **generate insights**—such as mood patterns, emotional trends, and common discussion themes—which will be displayed on the **psychologist's web portal**. This allows professionals to monitor patient progress and understand emotional states more effectively.

Core pieces of a RAG system

1. **Knowledge base** — the collection of text documents (your paraphrased DSM summaries, coping strategies, FAQs).
2. **Chunking** — split long documents into small pieces (200–500 tokens) so retrieval works well.
3. **Embedding model** — converts text into vectors (numbers) so similar texts are close together.
 - a. Example model: all-MiniLM-L6-v2 (good, small, fast).
4. **Vector store** — where vectors live (FAISS, Chroma, Milvus). It finds nearest neighbors.
5. **Retriever** — queries the vector store and returns top-k relevant chunks.
6. **Generator (LLM)** — the large language model that creates the final reply using the retrieved chunks.

7. **Prompt template / orchestration** — how you feed retrieved chunks + user message to the LLM.
8. **Safety & policy layer** — checks for crisis words, prevents harmful advice, escalates to humans.
9. **Logging & analytics** — save chat history for psychologist dashboard and to build insights.

How RAG will work in prototype — step by step

Here's a plain user-to-system flow for your mobile app:

1. **User types** (or speaks): “I’ve been feeling hopeless and can’t sleep.”
2. **System logs** the message and optionally does voice→text.
3. **Retriever**: converts the user text into an embedding and searches the vector store for the most relevant chunks (e.g., summaries about depression symptoms, sleep tips, crisis guidance).
4. **LLM Prompt**: you prepare a prompt that includes:
 - a. a short system instruction (be empathetic, non-judgmental),
 - b. the retrieved context (only a few short chunks),
 - c. the user message.
5. **Generator (LLM)**: produces an empathetic answer that references the retrieved context (e.g., coping tips, safe steps).
6. **Safety check**: if the user message contains suicide/self-harm signals, run crisis flow: show emergency resources and notify human clinician if required.
7. **Send reply** to the mobile app (text, and optionally TTS to speak).
8. **Store chat** for analytics — later the psychologist web portal can view mood trends, frequent topics, and flags.

Instead of reading and chunking the PDF every time, do this **one time** before starting the chatbot:

- Split the PDF into smaller **text chunks** (for example, 500–1000 words each).
- Generate **embeddings** (numerical vector form of each chunk).

- Save them in a **vector database** like:
 - **FAISS (lightweight and offline)**
 - **Chroma** (very simple and local)
 - **Pinecone / Weaviate** (if cloud-based)

✅ Benefit:

When the user asks a question, you instantly search in these embeddings — it's very fast!

Step	Problem	Solution
Retrieval too slow	PDF not preprocessed	Preprocess + use FAISS/Chroma
Embeddings heavy	Using large model	Use small embedding model
Too much context	Too many chunks	Limit top 3–5
LLM slow	Large model	Use GPT-3.5 or Phi-3-mini
Repeated questions	Recomputing again	Cache responses
Parallel tasks	Sequential processing	Async execution
Small talk	Still runs RAG	Use direct responses

Division of Work (Equal Backend Contribution)

Person 1 — Data & Knowledge Base Engineer (Preprocessing + Embeddings)

Main Role:

Responsible for preparing and structuring the knowledge base that powers the RAG (Retrieval-Augmented Generation) model.

Tasks:

1. PDF Extraction:

- Use tools such as pdfplumber or PyPDF2 to extract text from the DSM-5-TR PDF.
- Clean the extracted text by removing headers, footers, page numbers, and extra spaces.
- Save the cleaned text into a .txt file for further processing.

2. Text Chunking:

- a. Split the cleaned text into smaller, meaningful chunks (around 500–1000 tokens) using LangChain’s RecursiveCharacterTextSplitter.
- b. Experiment with different chunk sizes to balance retrieval accuracy and speed.

3. Embeddings Creation:

- a. Generate vector embeddings for each chunk using models such as SentenceTransformers (all-MiniLM-L6-v2) or OpenAIEmbeddings.
- b. Store these embeddings along with chunk metadata in a vector database like FAISS or ChromaDB.

4. Storage and Testing:

- a. Save the final vector database file (e.g., dsm_index.faiss).
- b. Create a test script to perform sample queries and verify that the most relevant chunks are retrieved correctly.

✅ Deliverables:

- A reusable vector database file ready for retrieval.
- A complete script (build_knowledge_base.py) to rebuild the database anytime.



Person 2 — Retrieval & Generation Engineer (Query Handling + GPT Integration)

Main Role:

Responsible for building the AI logic that retrieves relevant information from the knowledge base and generates meaningful responses using GPT-3.5 or a local model.

Tasks:

1. Load Existing Vector Database:

- a. Load the FAISS/Chroma database created by Person 1.
- b. Implement a function to perform semantic search:

```
def retrieve_context(query, top_k=3):  
    # Search in FAISS  
    return top_chunks
```

2. RAG Pipeline (Core Logic):

- a. Build a function `get_answer(query)` that:
 - i. Retrieves the most relevant text chunks from the vector database.
 - ii. Combines these chunks with the user query.
 - iii. Sends the combined input to GPT-3.5 to generate a final response.

3. Prompt Engineering:

- a. Design a clear and concise prompt for the model:

You are a helpful AI trained on mental health information.
Use the context below to answer the question accurately.

Context: {retrieved_text}

Question: {user_query}

4. Speed Optimization:

- a. Retrieve only the top 3 chunks for faster response times.
- b. Keep prompts short to minimize token usage.
- c. Implement caching if time allows to reduce repeated computation.

5. Streamlit Integration:

- a. Develop a simple Streamlit interface (`app.py`) to interact with the chatbot.
- b. Connect the interface to the RAG pipeline (`get_answer()` function) for real-time testing and demonstration.

✅ Deliverables:

- `rag_engine.py` (main AI logic file).
- A working chatbot demo through Streamlit that retrieves relevant context and generates accurate, fast responses.